CSCI 3104 Fall 2024
Instructors: Prof. Chen

Problem Set 8

Due Date ..... November 14
Name ..... **Hyma Jujjuru**
Student ID **110743269**
Collaborators ..... **Apollo Hoffman**

Contents

Instructions **1**

Honor Code (Make Sure to Virtually Sign) **2**

1 Dynamic Programming II **3**
1.1 Problem 1 [15 pts] 3
1.2 Problem 2 [15 pts] 4
1.3 Problem 3 [15 pts] 5
1.4 Problem 4 [55 pts + 15 extra pts] 6

Instructions

- The solutions **should be typed**, using proper mathematical notation. We cannot accept hand-written solutions. Here's a short intro to LaTeX.
- You should submit your work through the **class Gradescope page** only (linked from Canvas). Please submit one PDF file, compiled using this LaTeX template.
- You may not need a full page for your solutions; pagebreaks are there to help Gradescope automatically find where each problem is. Even if you do not attempt every problem, please submit this document with no fewer pages than the blank template (or Gradescope has issues with it).
- You are welcome and encouraged to collaborate with your classmates, as well as consult outside resources. You must **cite your sources in this document. Copying from any source is an Honor Code violation. Furthermore, all submissions must be in your own words and reflect your understanding of the material.** If there is any confusion about this policy, it is your responsibility to clarify before the due date.
- Posting to **any** service including, but not limited to Chegg, Reddit, StackExchange, etc., for help on an assignment is a violation of the Honor Code.
- You **must** virtually sign the Honor Code (see Section). Failure to do so will result in your assignment not being graded.

Honor Code (Make Sure to Virtually Sign)]

Problem HC.

- My submission is in my own words and reflects my understanding of the material.
- Any collaborations and external sources have been clearly cited in this document.
- I have not posted to external services including, but not limited to Chegg, Reddit, StackExchange, etc.
- I have neither copied nor provided others solutions they can copy.

Agreed (signature here). I agree to the above, Hyma Jujjuru. □

1 Dynamic Programming II

1.1 Problem 1 [15 pts]

Problem 1. Consider the following input for the Knapsack problem with a capacity $W = 11$:

item i	1	2	3	4	5	6
value v_i	2	6	19	24	29	34
weight w_i	1	2	5	6	7	8

and the corresponding table for the optimal values/profits:

	0	1	2	3	4	5	6	7	8	9	10	11
{ }	0	0	0	0	0	0	0	0	0	0	0	0
{ 1 }	0	2	2	2	2	2	2	2	2	2	2	2
{1, 2 }	0	2	6	8	8	8	8	8	8	8	8	8
{ 1, 2, 3 }	0	2	6	8	8	19	21	25	27	27	27	27
{1, 2, 3, 4 }	0	2	6	8	8	19	24	26	30	32	32	43
{1, 2, 3, 4, 5 }	0	2	6	8	8	19	24	29	31	35	37	43
{ 1, 2, 3, 4, 5, 6 }	0	2	6	8	8	19	24	29	34	36	40	43

Draw the backward path consisting of backward edges to find the subset of items that has the optimal value/profit. Besides indicating the backward path, you must also give the optimal-value subset of items. Clearly explain your work.

Answer. **Backward Path:**

	0	1	2	3	4	5	6	7	8	9	10	11
{ }	0	0	0	0	0	0	0	0	0	0	0	0
{ 1 }	0	2	2	2	2	2	2	2	2	2	2	2
{1, 2 }	0	2	6	8	8	8	8	8	8	8	8	8
{ 1, 2, 3 }	0	2	6	8	8	19	21	25	27	27	27	27
{1, 2, 3, 4 }	0	2	6	8	8	19	24	26	30	32	32	43
{1, 2, 3, 4, 5 }	0	2	6	8	8	19	24	29	31	35	37	43
{ 1, 2, 3, 4, 5, 6 }	0	2	6	8	8	19	24	29	34	36	40	43

Optimal-Value Subset of items: { item 3, item 4}
item 3 - { value: 19, weight: 5}
item 4 - { value: 24, weight: 6}

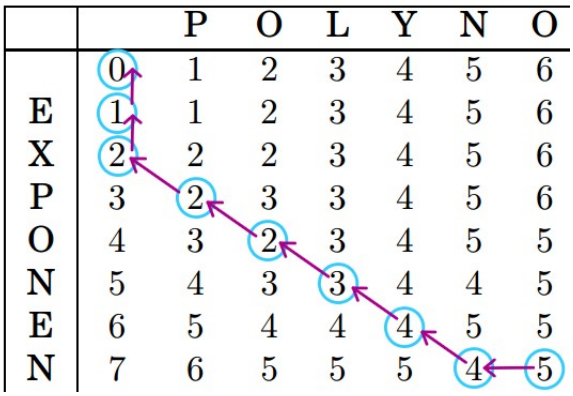
This subset is found through backtracking as shown above. Lets say the table above is T(i,j) such that is the rows of subsets and j is the columns of weight capacities. We start at T(6, 11) = 43 and move up to T(4, 11) = 43. Here we first add item 4 to the optimal solution set. Then we subtract the highest weight in the subset which is weight of item 4 (value: 24, weight: 6) from 11 and move up 1 to T(3, 5) = 19. The highest weight of this subset is item 3's weight 5 so we add item 3 to the optimal solution set and we subtract 5 from 5 and move up 1 to get T(2, 0) = 0. So we move up to T(0,0) = 0 and we are done backtracking. □

1.2 Problem 2 [15 pts]

Problem 2. Recall the sequence alignment problem where the cost of *sub* and the cost of *indel* are all 1. Given the following table of optimal cost of aligning the strings EXPONEN and POLYNO, draw the backward path consisting of backward edges to find the minimal-cost set of edit operations that transforms EXPONEN to POLYNO. Besides indicating the backward path, you must also give the minimal-cost set of edit operations. Clearly explain your work.

		P	O	L	Y	N	O
	0	1	2	3	4	5	6
E	1	1	2	3	4	5	6
X	2	2	2	3	4	5	6
P	3	2	3	3	4	5	6
O	4	3	2	3	4	5	5
N	5	4	3	3	4	4	5
E	6	5	4	4	4	5	5
N	7	6	5	5	5	4	5

Answer. **Backtracking**:



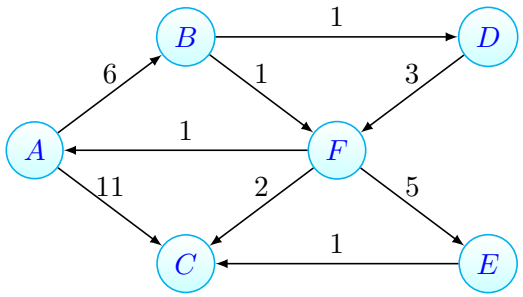
Minimum-Cost Operations From S(0,0) to S(7, 6):
vertical: *delete* operation
horizontal: *insert* operation
going up 1 and left 1 or 1 diagonal = *sub* operation
delete(1) $\longrightarrow$ *delete*(1) $\longrightarrow$ no op(0) $\longrightarrow$ no op(0) $\longrightarrow$ *sub*(1) $\longrightarrow$ *sub*(1) $\longrightarrow$ no op(0) $\longrightarrow$ *insert*(1) = 5
Minimum-Cost Set = { delete, delete, sub, sub, insert}

□

1.3 Problem 3 [15 pts]

Problem 3. Consider the weighted directed graph $G(V, E, w)$ pictured below. Work through the Bellman-Ford algorithm with the destination vertex C. Note that you must use the version of the algorithm presented in lecture; see also slide 14 of the in-class slides for Oct 29 & 31.

- Clearly specify the cost $d(v)$ of the current best path from each node $v \in V$ to C as well as the corresponding successor node at each iteration/pass. You may want to make a table to store the costs and successors.
- Give the shortest path tree, i.e., the union of all the shortest paths to C from all other vertices. For your convenience, you may want to modify the "latex code" for the given graph to draw the shortest path tree.



Answer. **Cost** $d(v)$ of current best path from each node $v \in V$ to C

Initialization:

Node	Cost $d(v)$	Successor
A	∞	NULL
B	∞	NULL
C	0	C
D	∞	NULL
E	∞	NULL
F	∞	NULL

1^{st} iteration:

Node	Cost $d(v)$	Successor
A	11	C
B	∞	NULL
C	0	C
D	∞	NULL
E	1	C
F	2	C

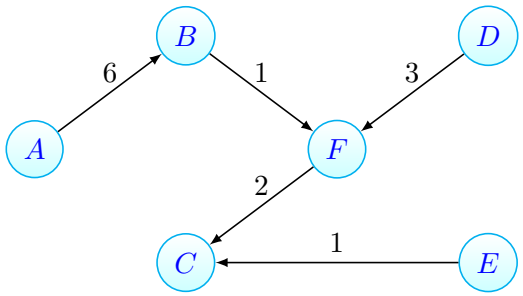
2^{nd} iteration:

Node	Cost $d(v)$	Successor
A	11	C
B	3	F
C	0	C
D	5	F
E	1	C
F	2	C

3^{rd} iteration:

Node	Cost $d(v)$	Successor
A	9	B
B	3	F
C	0	C
D	5	F
E	1	C
F	2	C

Shortest Path Tree



□

1.4 Problem 4 [55 pts + 15 extra pts]

Problem 4. Recall that the *string alignment problem* takes as input two strings x and y, composed of symbols $x_i, y_j \in \Sigma$, for a fixed symbol set Σ, and returns a minimal-cost set of *edit* operations for transforming the string x into string y.

Let x contain n_x symbols, let y contain n_y symbols, and let the set of edit operations be those defined in the lecture notes (substitution, insertion, deletion, and transposition).

Let the cost of *indel* be 1, the cost of *swap* be 5 (plus the cost of the two *sub* ops), and the cost of *sub* be 3, except when $x_i = y_j$, which is a "no-op" and has cost 0.

For this problem, please use the provided starter pseudocode to implement and apply these three functions:

(i) `alignStrings(x,y)` takes as input two ASCII strings x and y, and runs a dynamic programming algorithm to return the cost matrix S, which contains the optimal costs for all the subproblems for aligning these two strings.

```
alignStrings(x,y) :                    // x,y are ASCII strings
   S = table of length nx+1 by ny+1    // for memoizing the subproblem costs
   initialize S                        // fill in the base cases
   for i = 1 to nx
        for j = 1 to ny
             S[i,j] = cost(i,j)      // optimal cost for x[0..i] and y[0..j]
   }}
   return S
```

(ii) `extractAlignment(S,x,y)` takes as input an optimal cost matrix S, strings x, y, and returns a vector a that represents an optimal sequence of edit operations to convert x into y. This optimal sequence is recovered by finding a path on the implicit DAG of decisions made by `alignStrings` to obtain the value $S[n_x, n_y]$, starting from $S[0, 0]$.

```
extractAlignment(S,x,y) :  // S is an optimal cost matrix from alignStrings
   initialize a            // empty vector of edit operations
   [i,j] = [nx,ny]         // initialize the search for a path to S[0,0]
   while i > 0 or j > 0
        a[i]  = determineOptimalOp(S,i,j,x,y) // what was an optimal choice?
        [i,j] = updateIndices(S,i,j,a)        // move to next position
   }
   return a
```

When storing the sequence of edit operations in a, use a special symbol to denote no-ops.

(iii) `commonSubstrings(x,L,a)` which takes as input the ASCII string x, an integer $1 \leq L \leq n_x$, and an optimal sequence a of edits to x, which would transform x into y. This function returns each of the substrings of length at least L in x that aligns exactly, via a run of no-ops, to a substring in y.

1. (30 pts) From scratch, implement the functions `alignStrings`, `extractAlignment`, and `commonSubstrings`. You may not use any library functions that make their implementation trivial. Within your implementation of `extractAlignment`, ties must be broken uniformly at random.

   Submit (i) a paragraph for each function that explains how you implemented it (describe how it works and how it uses its data structures), and (ii) your code implementation, with code comments.

   Hint: test your code by reproducing the `APE` / `STEP` and the `EXPONENTIAL` / `POLYNOMIAL` examples in the lecture notes (to do this exactly, you'll need to use unit costs instead of the ones given above).

Answer.

- `alignStrings`
  The function creates a cost matrix S where S[i][j] represents the minimum cost to align the first i characters of x with the first j characters of y. It initializes the base cases for deleting and inserting, and then it fills the matrix by iterating through each character combination. The costs are determined based on whether the characters match or not ('no-op' or not) and based on the costs provided. The function returns the completed cost matrix.

```
noop = 0
insert = 1
delete = 1
sub = 3
swap = 5

def alignStrings(x, y):
    nx,ny = len(x), len(y)
    # Create the cost matrix S
    S = [[0] * (ny + 1) for _ in range(nx + 1)]

    # Initialize base cases for deletions and insertions
    for i in range(1, nx + 1):
        S[i][0] = i  # Cost of deleting all characters in x
    for j in range(1, ny + 1):
        S[0][j] = j  # Cost of inserting all characters in y

    # Fill in the cost matrix S
    for i in range(1, nx + 1):
        for j in range(1, ny + 1):
            # Cost for substitution (only if characters are different)
            if x[i - 1] == y[j - 1]:
                cost = noop
            else:
                cost = sub
            S[i][j] = min(
                S[i - 1][j] + delete,        # Deletion
                S[i][j - 1] + insert,        # Insertion
                S[i - 1][j - 1] + cost       # Substitution
            )
            # Check for potential swap
            if i > 1 and j > 1 and x[i - 1] == y[j - 2] and x[i - 2] == y[j - 1]:
                S[i][j] = min(S[i][j], S[i - 2][j - 2] + swap)  # Cost of swap

    return S
```

- `extractAlignment`
  The function takes the cost matrix S (produced from `alignStrings`), along with the original strings x and y, and constructs the sequence of edit operations required to transform x into y. Starting from the bottom-right of the matrix, it traces back through the operations made while filling the matrix. When multiple paths are possible, it breaks ties randomly to ensure uniformity. The operations 'NO-OP', 'DELETE', 'INSERT', 'SUBSTITUTE', and 'SWAP' are appended to the list a along with the corresponding x[i] character which will be used for determining common substrings, which is reversed at the end to provide the order of operations from from S[0,0] to S[nx, ny].

```
def extractAlignment(S, x, y): # S is an optimal cost matrix from alignStrings
    a = [] # empty vector of edit operations
    i, j = len(x), len(y) # initialize the searching indices for a path to S[0,0]

    while i > 0 or j > 0:
        choices = [] #tuple that will hold possible operations at each step

        # determine and append optimal choice.
        # If there are multiple, all operations with same cost are appended
        # Compare using the min cost formula based on operation values from class

        # Check for no-op
        if i > 0 and j > 0 and (x[i - 1] == y[j - 1] or S[i][j] == S[i - 1][j - 1] + noop):
            choices.append(("NO-OP", i - 1, j - 1))

        # Check for delete
        if i > 0 and S[i][j] == S[i - 1][j] + delete:
            choices.append(("DELETE", i - 1, j))

        # Check for insert
        if j > 0 and S[i][j] == S[i][j - 1] + insert:
            choices.append(("INSERT", i, j - 1))

        # Check for substitute
        if i > 0 and j > 0 and S[i][j] == S[i - 1][j - 1] + sub:
            choices.append(("SUBSTITUTE", i - 1, j - 1))

        # Check for swap
        if (i > 1 and j > 1 and x[i - 1] == y[j - 2] and x[i - 2] == y[j - 1]
        and S[i][j] == S[i - 2][j - 2] + swap):
            choices.append(("SWAP", i - 2, j - 2))

        # Randomly choose one of the valid operations from choices.
        # This is where we break ties and add to solution set
        if choices:
            operation, xi, yj = random.choice(choices)
            a.append((operation, x[xi] if operation != "INSERT" else y[yj]))

            #update indices
            if operation == "SUBSTITUTE" or operation == "NO-OP":
                i -= 1
                j -= 1
            elif operation == "DELETE":
                i -= 1
```

```
        elif operation == "INSERT":
            j -= 1
        elif operation == "SWAP":
            i -= 2
            j -= 2

a.reverse()  # Reverse to have the operations in the correct order from S[0,0] to S[nx, ny]
return a
```

- `commonSubstrings`
  The function identifies substrings in x that align with substrings in y through 'NO-OP' operations. It takes the string x, an integer L that denotes the minimum length of the substrings, and the list of edit operations. By iterating through the list of edit operations, it appends characters from x during the 'NO-OP' operations. If it finds a different operation, it checks if the appended substring meets the minimum length requirement before resetting the temporary list. It returns a list of unique substrings that meet the length criteria.

```
def commonSubstrings(x, L, a):
    common_subs = set()
    current = []

    for operation, ch in a:
        if operation == 'NO-OP':
            current.append(ch)  # add the character to the substring
        else:
            if(len(current) >= L):
                common_subs.add(''.join(current))
                current = []


    if(len(current) >= L):
        common_subs.add(''.join(current))

    return list(common_subs)
```

2. (10 pts) Using asymptotic analysis, determine the running time of the call
`commonSubstrings(x, L, extractAlignment( alignStrings(x,y), x,y ) )`
Justify your answer.

Answer.
The asymptotic runtime of `alignStrings(x,y)` is $O(nx \cdot ny) = O(n^2)$ because there a 2 single forloops and a double for loop that loops through the 2D matrix S.

The asymptotic runtime of `extractAlignment` is $O(nx + ny) = O(n)$ as there is a single while loop which means the worst case time complexity is running the loop n times.

The asymptotic runtime of `commonSubstring` is $O(n \cdot n + n) = O(n^2)$ as there is a single for loop that runs through a produced by `extractAlignment` which means the worst case time complexity O(n) is running the loop n times. But during the loop if the join() is run then its time complexity is O(n) as it can run a worst case of n times. So the combined time complexity is $O(n^2)$

As the three functions run linearly and one after the other as `extractAlignment` is reliant on output form `alignStrings` and `commonSubstrings` relies on output from `extractAlignment`. So the asymtotic analysis of the call `commonSubstrings(x, L, extractAlignment( alignStrings(x,y), x,y ) )` is $O(n^2)$ as that is the upper bound between n^2, n, and n^2.

3. (15 pts extra credit) Describe an algorithm for counting the number of optimal alignments, given an optimal cost matrix S. Prove that your algorithm is correct, and give is asymptotic running time.

   Hint: Convert this problem into a form that allows us to apply an algorithm we've already seen.

   Answer.
   This is similar to how path counting works. We are trying to backtrack the cost matrix produced by the `alignStrings` function. To find the number of min-cost paths to the bottom-right cell from S[0,0].

   Below is a pseudocode for a function `countOptimalAlignments` which takes in the parameters S, the cost matrix, and the original strings, x and y. The function also keeps track of indices i and j of the cost matrix S for recursion purposes. The function is very similar in logic to the `extractAlignments` from part 1.

   Pseudocode:

```
CountAlignmentsRecursive(S, x, y, i, j):
    // initialize a DP table
    T = [] // will add pairs of (i,j)

    // Base case: If both indices are zero, there is one way (two empty sequences)
    if i == 0 and j == 0:
        Return 1

    // Check if the result is already computed
    if T[i,j] already stored:
        Return T[i, j]

    // Initialize ways to count the number of alignments
    count = 0

    // Check for NO-OP
    if i > 0 and j > 0 and (x[i - 1] == y[j - 1] or S[i][j] == S[i - 1][j - 1] + noop):
        count += CountAlignmentsRecursive(S, x, y, i - 1, j - 1)

    // Check for DELETE
    if i > 0 and S[i][j] == S[i - 1][j] + delete:
        count += CountAlignmentsRecursive(S, x, y, i - 1, j)

    // Check for INSERT
    if j > 0 and S[i][j] == S[i][j - 1] + insert:
        count += CountAlignmentsRecursive(S, x, y, i, j - 1)

    // Check for SUBSTITUTE
    if i > 0 and j > 0 and S[i][j] == S[i - 1][j - 1] + sub:
        count += CountAlignmentsRecursive(S, x, y, i - 1, j - 1)

    // Check for SWAP
    if(i > 1 and j > 1 and (x[i - 1] == y[j - 2]
    and x[i - 2] == y[j - 1]) and S[i][j] == S[i - 2][j - 2] + swap):
        count += CountAlignmentsRecursive(S, x, y, i - 2, j - 2)
    // Store the count in the DP table for future use
    T[i,j] = count

    // Return the total number of ways to reach S[i][j]
    Return count
```

Proof of Correctness by Induction:
Base Case:
When i = 0 and j = 0, then the strings x and y are empty so there is only one way to arrange them.

Inductive Hypothesis:
Assume that for all pairs of (a, b) such that $0 \leq a \leq i$ and $0 \leq b \leq j$, the function returns the correct number of optimal alignments for the first i characters of x and teh first j characters of y.

Inductive Step:
We can use the min cost formula from class notes from Nov 5 to check whether the minimum cost operation at the step is 'NO-OP', 'INSERT', 'DELETE', 'SUBSTITUTE', or 'SWAP'. We need to show that the inductive hypothesis hold for (i,j)

- NO-OP: If x[i-1] = y[j-1] or the cost S[i][j] matches the cost of S[i - 1][j - 1] + noop, then: CountAlignmentsRecursive(S, x, y, i-1, j-1) counts the alignments that result from aligning the last characters of both sequences with no operation, contributing to the count.
- DELETE: If the cost S[i][j] matches the cost of S[i - 1][j] + delete then: CountAlignmentsRecursive(S, x, y, i - 1, j) counts the number of ways to align the first i - 1 characters of x with the first j characters of y when the character x[i-1] is deleted.
- INSERT: If the cost S[i][j] matches the cost of S[i][j - 1] + insert then: CountAlignmentsRecursive(S, x, y, i, j - 1) counts the number of ways the first i characters of x with the first j - 1 characters of y when the character x[i] is inserted.
- SUBSTITUTE: If the cost S[i][j] matches the cost of S[i - 1][j - 1] + sub then: CountAlignmentsRecursive(S, x, y, i - 1, j - 1) counts the number of ways the first i - 1 characters of x with the first j - 1 characters of y after substituting.
- SWAP: If the two characters can be swapped, then: CountAlignmentsRecursive(S, x, y, i - 2, j - 2) counts the number of ways the two characters are swapped.
- Final Count: the total number of optimal alignments is found by adding up the counts from each scenario above.

So, by induction, the function returns the correct number of optimal alignments, given an optimal cost matrix S.

Asymptotic Running Time:
Each pair (i,j) is called only once because they are stored in the DP table T for future use. This means that worst case, i will go from 0 to n and j will go from 0 to m, which means there are a total of $n \cdot m$ possible pairings of i and j.
The total asymptotic running time analysis is $O(n \cdot m) = O(n^2)$

4. (15 pts) String alignment algorithms can be used to detect changes between different versions of the same document (as in version control systems) or to detect verbatim copying between different documents (as in plagiarism detection systems).

   The two `data_string` files for PS8 (see class Canvas) contain actual documents recently released by two independent organizations. Use your functions from (1) to align the text of these two documents. Present the results of your analysis, including a reporting of all the substrings in x of length $L = 10$ or more that could have been taken from y, and briefly comment on whether these documents could be reasonably considered original works, under CU's academic honesty policy.

   Answer.
   Common Substrings of length at least 10:
   's ratey stin', 'expected to ', ' under a a', 'bi o o ica', 'ti that il', 'ast 2022. ', 'aesl st gi', ' oice o th', 'ullte bs th', ' adcixeefc', ' ress erta', 'n n uo, i', 'e reate t ', 'laed e l c', ' e new ive', 'n om whnon', 'investments ', 'nocoe d re', 'ching and l', 'te pc oay t', 'cnyis eay ', 'inn t s a', 'l eetedo t', 'de otet h', 'e nie sts ', ' opeating ', 't l o e t ', ' eate more', 'nd higpyng', 'e texas an', 'tes exxon mobil ', 'n the ommni', 'onastig ne', 'vnment and ', 'a. this a ', 'began in 20', 'wt aoch n a', 'es ch pr', 'a rn w in', ' al euaoen', ' e ancture', ' rant and l', 'ese a ullti', 'gay ore o i', 's along th', 'to dio o', 'y prve ine', 'raion oai ', '0 lion iet', ' reain oe ', 'etnts o t ', 'mi-mnri oe', ' than nst', 'ae ee a og', ' nd he annu', ',n t nlte ', 'sting o owt', ' xxnm nnon', ' ect ceatn', 'nt sd man ', ' de a os ', 'd louisian', '. on oili', '13 and are ', 'hae far-re', 'ty the nt ', 'xp ts ctra', 'es otisig the', 'rt o arreg', ' rea ae ai', ' il an ga', 'at mature ', 're eeted t', ' theevinment', 'ent ora a i', 'rojects at', 'thimntin su', ' that seic', ' .up oacnu', 'ore than ', ' said s the', 'tueaeicnse', ' js the ted ', 'e manucng ', 'e he pays ', 'rcionjos d', 'o t roha w', 'n o v ,0ry', 't te ted s', 'iquefied n', 'at , prsie', 'th ile ect', ' planne or', 'salai ngin', ' facilitie', 'd ere in t', 'mensnnce ', 'il's projects', ' pedent tu', ' proposed new ', 'os that e m', 'th whi hos', 'hese s l a', ' poa nit 1', 'sisty cere', 'continue thr', 'eppumrian ', 'e a mutilie', ', once com', ' or ay the', 'it mp onatu', 'atural gas p', 'ha00 crtio', ' inefninae', 'e ontris m', 'pected to ', 'levels, are ex', ',c-ein oal', ' as gig h g', 'ts projects', 'ronou aa tr', 'eng rys pro', 'ty hhslld ', ' exxon mob', 'a ing a ey', ' ad mnauin', 'a coasts. ', 'r fr ieite ', 'and existing', ' eric wor ', ' j-ratin v', ' t ae aeyon', 'pleted and', 'intent prg', ' fro5 t t', 'aon pport ', 'mar healr,', 'ough at le'

   I believe that thses documents cannot be reasonably considered original works, under CU's academic honesty policy as there are 1452 no-op operations which means 1452 out of the 2558 characters in string x are common between the string x and string y.